

يعني إيه File Structure ؟

الـ File Structure ببساطة هي طريقة ترتيب الملفات والمجلدات (Folders) في أي مشروع. الهدف منها إنك تخلي مشروعك منظم وسهل التعامل، سواء ليك أو لأي حد شغال معاك.

ليه مهم نرتب الملفات؟

1. **سهولة الفهم:** لما الملفات تكون مترتبة، أي حد يفتح المشروع يعرف كل حاجة فين من غير ما يتوه.
2. **توفير وقت:** بدل ما تقعد تدور على ملف معين وسط كركبة، هتلاقيه بسهولة.
3. **سهولة التعديل:** لو عايز تضيف ميزة أو تعدل حاجة، هتعرف توصل للمكان الصحيح بسرعة.
4. **تعاون أسهل:** لو شغال مع فريق، الكل هيبقى فاهم المشروع ومش هيحصل لخبطه.

1. Flat Structure (الهيكلية المسطحة): كل الملفات في مجلد واحد

- يعني إيه؟ كل حاجة في ملف واحد أو مجلد واحد، زي HTML، CSS، JavaScript، والصور.
- مميزاتاها: سريعة وسهلة، ومش محتاجة ترتيب كثير.
- مشاكلها: لو المشروع كبير، هتبقى الدنيا زحمة وصعب تدور على حاجة.
- تستخدم إمتى؟ لو المشروع صغير جدًا، زي صفحة واحدة أو لعبة بسيطة جدًا.
- لو شغال لوحك أو عايز تعمل Prototype سريع.

```
project/  
├ index.html  
├ styles.css  
├ script.js  
├ image1.png  
└ sound1.mp3
```

2. Layered Structure (الهيكلية بالطبقات): الملفات بتتوزع حسب النوع (HTML - JS - CSS) و الـ assets

- يعني إيه؟ بتقسم المشروع على حسب نوع الملفات. الصور والصوتيات في فولدر، الأكواد (CSS و JavaScript) في فولدرات لوحدها.
- مميزاتاها: بتنظم المشروع أكثر، خصوصًا لو فيه شوية حاجات كثير.
- سهل تدور على حاجة معينة زي الصور أو الأكواد.
- تستخدم إمتى؟ لو المشروع صغير أو متوسط الحجم.
- لو الفريق صغير ويبشغل على حاجة واحدة زي لعبة صفحة واحدة.

```
project/  
├ assets/  
│   ├── images/  
│   └── audio/  
├ fonts/  
├ css/  
│   └── styles.css  
├ js/  
│   ├── main.js  
│   ├── game.js  
│   └── utils.js  
└ index.html
```

3.Feature-Based Structure (الهيكلية حسب المميزات):

الملفات بتتوزع على حسب مميزات المشروع

• يعني إيه؟

بتقسم المشروع على حسب المميزات أو الأجزاء بتاعته. مثلاً:

- كل ملفات اللاعبين (Player) مع بعضها.
- كل ملفات الأعداء (Enemies) مع بعضها.
- كل ملفات المستويات (Levels) مع بعضها.

• مميزاتها:

- سهلة لو كل واحد في الفريق شغال على جزء معين.
- المشروع بيبقى Modular (يعني كل جزء مستقل ومنظم).
- مثالية لو المشروع هيكبر أو هيبقى فيه مستويات وشخصيات كتير.

• تستخدم إمتى؟

- لو المشروع كبير وفيه أكثر من جزء (زي شخصيات، أعداء، مستويات).
- لو الفريق متقسم وكل واحد شغال على جزء معين.

```
project/
├── features/
│   ├── player/
│   │   ├── player.js
│   │   ├── player.css
│   │   └── player.html
│   ├── enemies/
│   │   ├── enemy.js
│   │   ├── enemy.css
│   │   └── enemy.html
│   └── levels/
│       ├── level1.js
│       └── level2.js
├── assets/
│   ├── images/
│   ├── audio/
│   └── fonts/
└── index.html
```

4.Test-Driven Structure (الهيكلية المبنية على الاختبارات):

```
project/
├── src/                <-- هنا الكود الأساسي
│   ├── assets/
│   ├── css/
│   ├── js/
│   │   └── game.js
│   └── index.html
├── tests/              <-- هنا ملفات الاختبارات
│   ├── unit/           <-- اختبارات لكل جزء صغير (زي حركة لاعب)
│   ├── integration/    <-- اختبارات لتأكد إن الأجزاء شغالة مع بعض
│   └── e2e/            <-- اختبارات لتجربة اللعبة ككل
└── .gitignore
```

لا يتم رفعه على الـ main

• يعني إيه؟

بتركز على إنك تعمل اختبارات (Tests) للكود بتاعك عشان تتأكد إنه شغال من غير مشاكل. الملفات بتنقسم لقولدين:

- واحد للكود الأساسي (src).
- واحد للاختبارات (tests).

• مميزاتها:

- بتقلل الأخطاء والعيوب في المشروع.
- تضمن إن الكود بيشغل كويس قبل ما حد يستخدمه.
- تستخدم إمتى؟
- لو الفريق بيشغل بأسلوب (Test-Driven Development (TDD.
- لو اللعبة معقدة زي ألعاب Multiplayer وعازين تتأكد إنها شغالة 100%.

يعني إيه بكل اختصار (TDD) Test-Driven Development ؟

تخيل إنك بتبني لعبة، بس بدل ما تبدأ تكتب الكود على طول (زي حركة اللاعب أو هجوم العدو)، لا، الأول بتكتب اختبار بسيط (Test) عشان تتأكد إن الكود اللي هتكتبه هيشغل صح.

خطوات TDD :

1. اكتب اختبار (Test) الأول:
يعني مثلاً اكتب اختبار بسيط يقول:
"لو اللاعب ضغط سهم فوق، المفروض يتحرك لفوق".
 2. اكتب الكود عشان يعدي الاختبار:
تبدأ تكتب كود حركة اللاعب اللي بيخليه يتحرك لما يضغط سهم فوق.
 3. شغل الاختبار:
لو الكود شغال صح، الاختبار هينجح.
لو مش شغال، الاختبار هيفشل، وساعتها تعرف فين المشكلة وتصلحها.
-